

Shearwater (Perdix) -> Erebus Data Mapping via

libdivecomputer

How libdivecomputer decodes a Shearwater Perdix dive, why a download can return only summary fields (time / duration / depth) without the minute-by-minute profile, and how each decoded value maps to the Erebus dive_logs / dive_log_samples schema.

Reference source (authoritative): `src/shearwater_predator_parser.c` (the Shearwater Predator/Petrel parser). Erebus schema: `server/services/diveImportDTO.js`, `server/services/diveLogPersistence.js`.

1. Root cause: libdivecomputer has two separate extraction APIs

When a downloaded dive is parsed, the data comes out through two completely different calls that do not overlap:

API call	What it returns	Maps to
<code>dc_parser_get_field()</code>	Summary only - divetime, max depth, gas mixes, tanks, atmospheric pressure, dive mode, deco model	dive_logs header row
<code>dc_parser_samples_foreach()</code>	The minute-by-minute profile - every sample's depth / temp / ppO2 / NDL / deco / CNS / tank pressure	dive_log_samples rows

Symptom -> cause: "We get basic time, duration, depth, but none of the minute-by-minute data" is exactly what happens when the integration calls `get_field()` (and `get_datetime()`) but never registers a sample callback and calls `dc_parser_samples_foreach()` - or calls it but discards the callback output. The profile is never emitted by `get_field`; it only ever arrives through the `samples_foreach` callback.

UDDF import works because the UDDF importer (`parseUDDFSamples -> dto.addSample(...)`) explicitly walks every `<waypoint>`. The libdivecomputer path needs the equivalent loop over `samples_foreach`.

2. How samples_foreach emits a sample

It does NOT hand over one struct per sample. It calls the callback multiple times per time-step, once per field, all sharing the same `sample.time`.

Accumulator pattern (standard Subsurface / dctest approach):

- On `DC_SAMPLE_TIME` -> flush the previous sample, start a new one.
- On every other `DC_SAMPLE_*` -> set a field on the current sample.

Callback -> Erebus dive_log_samples mapping

Erebus stores `sample_time_seconds`, `depth_meters`, `temperature_celsius`, plus a metrics JSON blob, with tank pressures and events in side tables.

libdivecomputer callback	Value (SI; converted by parser)	Erebus column / metrics key
<code>DC_SAMPLE_TIME</code>	seconds	<code>sample_time_seconds</code>
<code>DC_SAMPLE_DEPTH</code>	metres	<code>depth_meters</code>
<code>DC_SAMPLE_TEMPERATURE</code>	C	<code>temperature_celsius</code>
<code>DC_SAMPLE_PPO2</code>	bar (<code>.value</code> ; <code>.sensor</code>)	<code>metrics.ppo2_bar</code>
<code>DC_SAMPLE_SETPPOINT</code>	bar	<code>metrics.setpoint_bar</code>
<code>DC_SAMPLE_CNS</code>	fraction (x100 for %)	<code>metrics.cns_percent</code>
<code>DC_SAMPLE_DECO (type=NDL)</code>	<code>.time</code> seconds	<code>metrics.ndl_seconds (/60 -> ndl_min)</code>
<code>DC_SAMPLE_DECO (type=DECOSTOP)</code>	<code>.depth</code> m, <code>.time</code> s, <code>.tts</code> s	<code>metrics.stop_depth_m</code> , <code>metrics.tts_seconds</code> , -> <code>metrics.ceiling_meters</code>
<code>DC_SAMPLE_GASMIX</code>	gas index	<code>event gas_switch + gas_slot</code>
<code>DC_SAMPLE_PRESSURE</code>	<code>.tank</code> , <code>.value</code> bar	<code>dive_log_tank_pressures</code> row (<code>gas_slot</code> , <code>pressure_bar</code>)

libdivecomputer callback	Value (SI; converted by parser)	Erebus column / metrics key
DC_SAMPLE_RBT	minutes	metrics (gas time remaining)
DC_SAMPLE_BEARING	degrees	metrics.bearing_deg
DC_SAMPLE_EVENT	bookmark / alarm	dive_log_events row

3. Raw byte layout - Perdix (Petrel Native Format)

The Perdix is a Petrel-family device, so:

- samplesize = 0x20 (32 bytes) per record.
- PNF (Petrel Native Format) is active when the first two bytes != 0xFFFF. In PNF, pnf = 1, so the record-type byte is at data[offset] and every field offset below is shifted by +1 (this is what the "+ pnf" in the source means).

Sample interval: logversion >= 9 -> uint16_be(opening[5] + 23) ms; otherwise the default is 10000 ms (10 s). Time accumulates += interval on each dive-sample record - it is NOT stored per sample.

Record type at data[offset]:

Type	Meaning
0x01	dive sample
0x02	freedive sample (4x 8-byte sub-samples packed in 32 bytes)
0x03	Avelo sample
0xE1	extended record (extra tank pressures)
0x30	info / bookmark event
0xFF	final record

All-zero records are skipped.

0x01 dive-sample record (offsets relative to offset + pnf)

Bytes	Field	Conversion
+0..1	depth	uint16_be / 10 -> m (metric)
+2..3	deco stop depth	uint16_be m; 0 => NDL
+4..5	TTS	uint16_be x 60 -> s
+6	ppO2 (CC, internal)	/100 -> bar
+7	O2 %	gas-change detection
+8	He %	gas-change detection
+9	deco stop time	x 60 -> s
+11	status flags	bit OC (0x10) clear => CCR; also SC, PPO2_EXTERNAL, SETPOINT_HIGH
+12 / +14 / +15	external ppO2 sensors 0 / 1 / 2	x calibration[n] (only if calibrated)
+13	temperature	signed char C (neg-fix: if <0, +=102, clamp <=0) - whole C, NOT Kelvin
+18	setpoint (Petrel)	/100 -> bar
+19 & +27	tank pressure (logv >= 7)	low 12 bits x 2 psi; >= 0xFFF0 = special code, ignore
+21	RBT / gas time	minutes (>= 0xF0 = special code, ignore)
+22	CNS (Petrel)	/100 -> fraction

Header / summary (PNF reads from opening[] / closing[] blocks)

Field	Source	Conversion
-------	--------	------------

Field	Source	Conversion
datetime	uint32_be(opening[0] + 12)	unix ticks
divetime	uint24_be(closing[0] + 6)	seconds
max depth	uint16_be(closing[0] + 4)	/10 -> m
gas mixes	O2 / He bytes	percentages
GF low / high	opening[0] + 4	values

4. Important: this does NOT match the in-app TS decoder

The homegrown `services/protocols/shearwaterProtocol.ts` uses a different and incorrect sample layout (it reads depth at +2, temperature as `uint16 / 10 - 273.15` Kelvin at +8, `ppO2` at +10, etc.). The real Perdix PNF format is the table in section 3 - temperature is a signed byte in whole C at +13, depth is at +0, and so on.

That mismatch is why the original react-plugin path produced wrong / empty profile data. `src/shearwater_predator_parser.c` is the authoritative layout - trust it over the TS file.

5. Bottom line

The profile is missing because the `libdivecomputer` integration is not consuming `dc_parser_samples_foreach()`. To fill `dive_log_samples` the way UDDF does:

1. Register a `dc_sample_callback_t`.
2. Accumulate one sample per `DC_SAMPLE_TIME` boundary.
3. Map each `DC_SAMPLE_*` to the columns / metrics keys in section 2.